



SWEET1N

QR-Powered Hotel & Restaurant Management System

Architecture & Technology Stack Review

Version 1.0

Prepared by	SpectreTech Limited
Prepared for	Developer Stephen Machera
Purpose	Architecture Review & Technical Recommendations
Date	July 2026

1. Overview

After reviewing the proposed architecture for the SWEET1N QR-Powered Hotel & Restaurant Management System, we believe the proposed solution provides a strong foundation for a modern restaurant management platform. The selected technologies are capable of delivering a responsive customer experience, real-time order management, and scalable cloud infrastructure.

However, several improvements are recommended to improve long-term scalability, maintainability, reporting capabilities, and future expansion into a multi-branch SaaS platform.

The recommendations below are intended to strengthen the architecture rather than replace it entirely.

2. Current Proposed Stack

Layer	Current Technology
Frontend	Next.js
Backend	Firebase Cloud Functions
Database	Cloud Firestore
Authentication	Firebase Authentication
Storage	Firebase Storage
Hosting	Firebase Hosting / Vercel
Payments	Stripe
AI	Claude API
Charts	Recharts

3. Overall Assessment

The proposed architecture is well suited for:

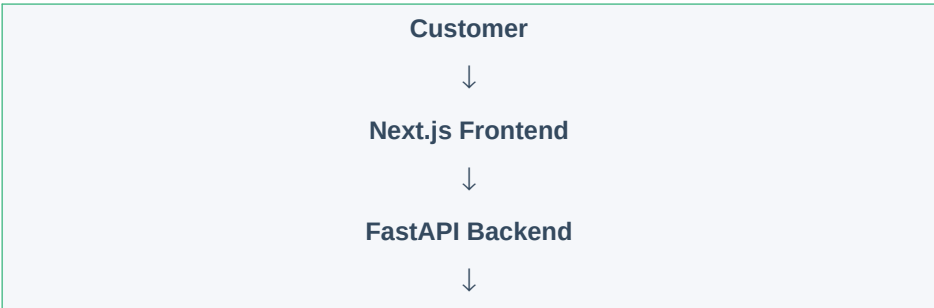
- Small restaurants
- Single-branch businesses
- Rapid MVP development
- Applications requiring real-time updates

Overall Rating: 8.5 / 10

While the architecture is technically sound, several components may become limiting as the system grows.

4. Recommended Architecture

Instead of placing all business logic inside Firebase Cloud Functions, we recommend introducing a dedicated backend service.





5. Technology Recommendations

Frontend

Recommendation: Keep Next.js

Reason:

- Excellent SEO
- Fast page loading
- Server-side rendering
- Progressive Web App support
- Modern React ecosystem
- Excellent dashboard routing

No changes recommended.

Backend

Current: Firebase Cloud Functions

Recommendation: Replace primary business logic with FastAPI (Python)

Firebase Cloud Functions should remain only for event triggers, scheduled jobs, push notifications, and Firebase-specific integrations.

Why? As the system grows, business logic such as order processing, inventory management, reservation handling, analytics, payments, and reporting becomes difficult to maintain when spread across many Cloud Functions. A centralized backend provides:

- Better maintainability
- Easier debugging
- Cleaner architecture
- Easier testing
- Better API versioning

Database

Current: Cloud Firestore

Recommendation: Replace Firestore with PostgreSQL

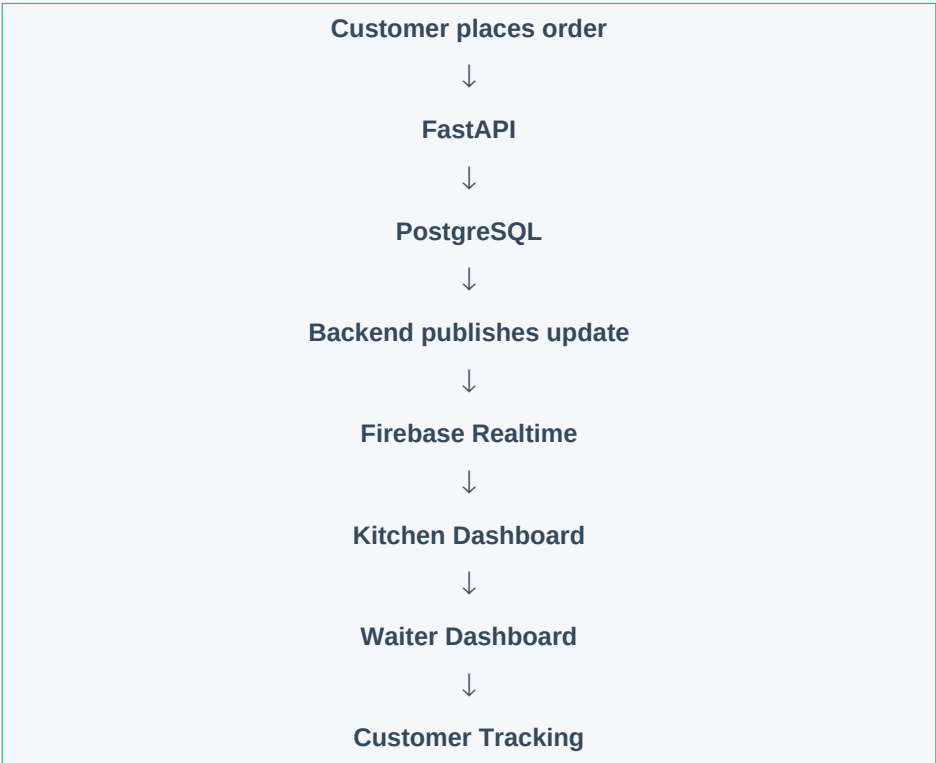
Restaurant systems are highly relational. Relationships include orders, order items, tables, customers, staff, transactions, reservations, inventory, branches, and reports. PostgreSQL is designed specifically for relational data and provides:

- JOIN operations
- Transactions
- Foreign keys
- Better reporting
- Faster analytics
- Strong consistency

Firestore remains excellent for real-time synchronization but is less suitable as the primary transactional database.

Realtime Updates

Continue using Firebase for real-time updates. Instead of storing all business data in Firestore:



This preserves Firebase's real-time advantages while keeping PostgreSQL as the system of record.

Cache Layer

Recommendation: Introduce Redis.

Redis should handle the active kitchen queue, session storage, frequently accessed menus, rate limiting, cached analytics, and temporary order state. Benefits include reduced database load, faster response times, and improved scalability.

Storage

Current: Firebase Storage | Recommendation: Keep Firebase Storage

It is suitable for menu images, staff photos, and documents. No changes required.

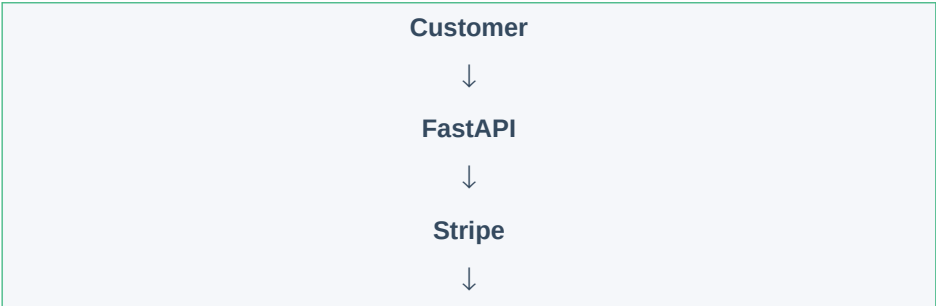
Authentication

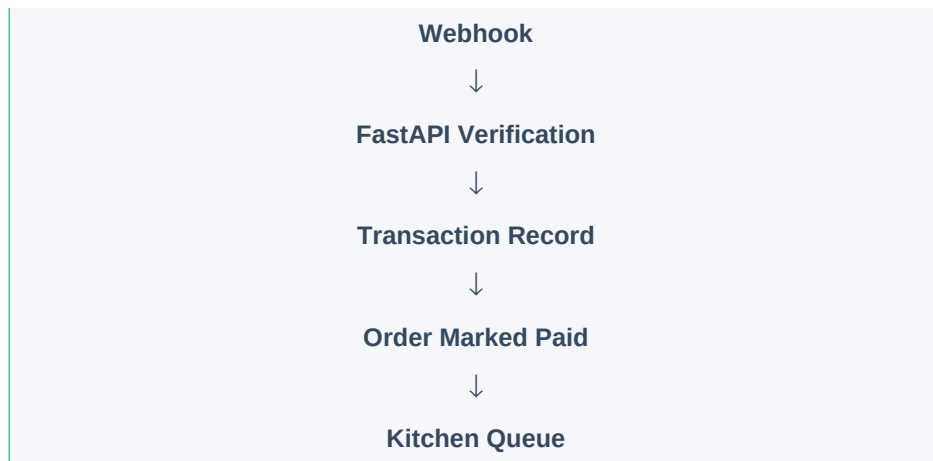
Current: Firebase Authentication | Recommendation: Keep Firebase Authentication (Supabase Authentication is a suitable alternative).

Payments

Current: Stripe | Recommendation: Keep current payment architecture

Recommended flow:



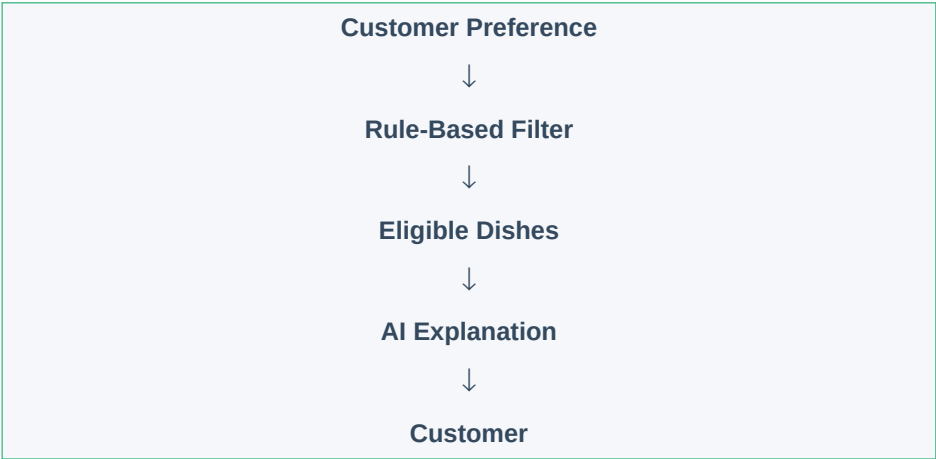


Every transaction should remain independent of the order for auditing purposes.

AI

Current: Claude API | Recommendation: Continue using AI only for explanations

Do NOT allow AI to determine food eligibility. Recommended flow:



This reduces AI cost while increasing reliability.

Hosting

Current: Firebase Hosting

Recommendation:

- Frontend → Vercel
- Backend → Docker container hosted on Railway, Hetzner VPS, DigitalOcean, HostAfrica VPS, or AWS Lightsail
- Database → Managed PostgreSQL

This provides better flexibility and avoids complete dependency on one cloud provider.

6. Additional Modules Recommended

The following modules are recommended for future versions.

Inventory Management

Automatic ingredient deduction after every order. Example: a Burger deducts bread, patty, cheese, and tomato — inventory is updated automatically.

Reservation Management

Allow customers to reserve tables, select date/time, specify guest count, and receive confirmations.

Kitchen Display System

Dedicated kitchen display for food preparation — larger display, better visibility, faster workflow.

Receipt Printing

Support for kitchen printers, cashier printers, and customer receipts.

Push Notifications

Notify customers when an order is accepted, preparing, ready, or served — across Push Notifications, SMS, and WhatsApp.

Offline Support

Support temporary internet outages using a Progressive Web App, IndexedDB, and Background Sync.

7. Identified Weaknesses

Vendor Lock-In

The current architecture relies heavily on Firebase. Future pricing or platform changes could affect the entire system. Introducing FastAPI and PostgreSQL reduces dependency on a single vendor.

Reporting

Firestore is not optimized for complex analytical queries. Restaurant analytics typically require revenue trends, top-selling products, staff performance, and inventory reports — significantly easier using SQL.

Data Relationships

Restaurant systems contain many interconnected entities — orders, order items, payments, inventory, tables, reservations, customers, and branches. Relational databases are better suited for these structures.

Maintainability

As the application grows, managing dozens of Cloud Functions becomes increasingly difficult. A centralized backend simplifies maintenance and testing.

8. Recommended Production Stack

Layer	Recommended Technology
Frontend	Next.js
UI	Tailwind CSS + shadcn/ui
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy + Alembic
Cache	Redis
Authentication	Firebase Authentication
Realtime	Firebase Firestore / Realtime Database (events only)
Storage	Firebase Storage
Payments	Stripe
AI	Claude API / OpenAI API
Deployment	Docker
Reverse Proxy	Nginx
CI/CD	GitHub Actions
Monitoring	Sentry + Grafana
Hosting	Vercel (Frontend) + VPS (Backend)

9. Final Recommendation

The current Firebase-based architecture is an excellent choice for rapidly delivering an MVP or supporting a single restaurant with moderate traffic. However, for a commercial-grade platform intended to support multiple branches, advanced analytics, inventory management, reservations, and long-term scalability, a hybrid architecture centered on **FastAPI**, **PostgreSQL**, and **Redis**, while retaining **Firebase** for authentication, storage, and real-time event delivery, provides a more robust and maintainable foundation.

This approach combines the strengths of relational data modeling, centralized business logic, and low-latency real-time updates, positioning the SWEET1N platform for sustainable growth and future expansion into a multi-tenant SaaS solution.

Prepared by SpectreTech Limited — Reliable Tech Solutions